

Welcome to Week 6

APIs and Web Scrapping

APIs

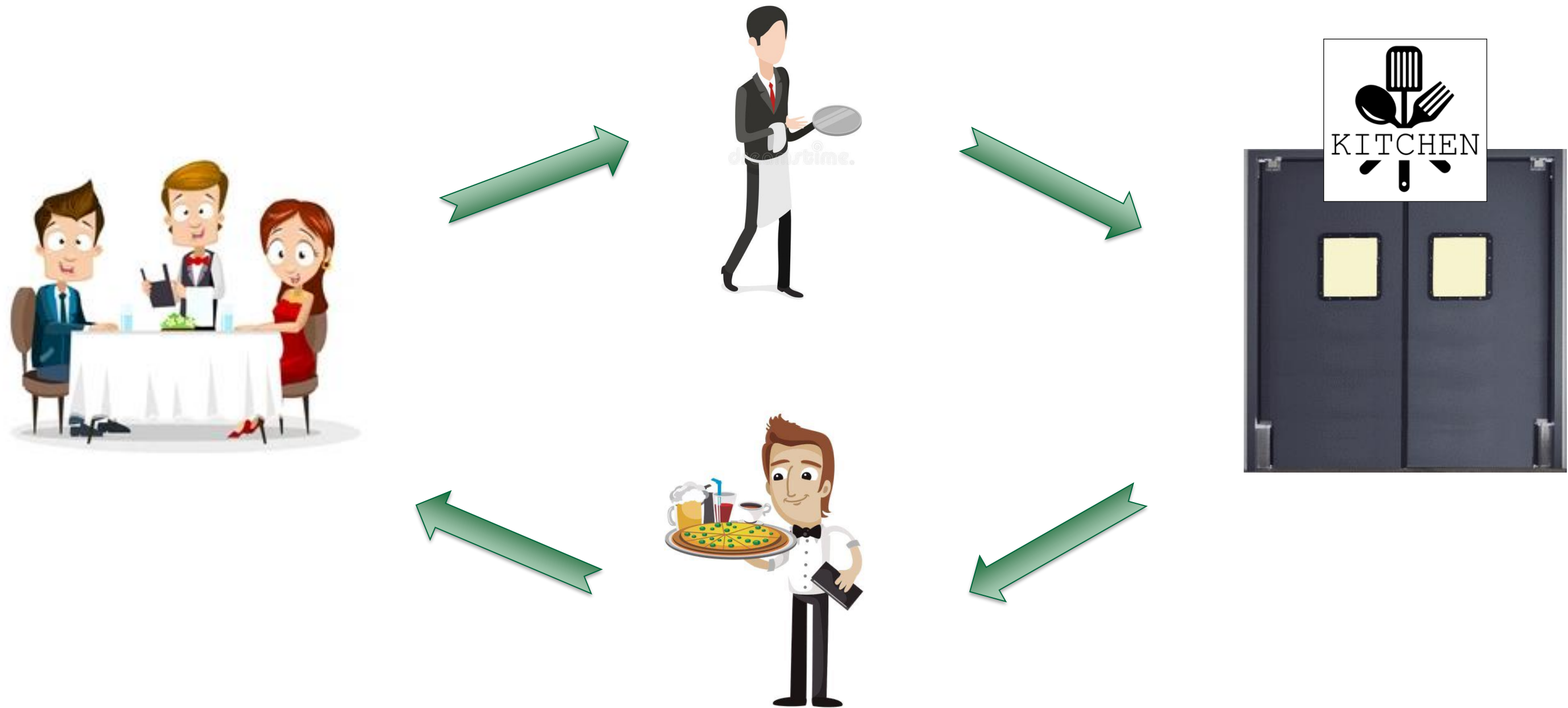
Application Programming Interface (API)

API stands for Application Programming Interface and refers to a set of defined rules and protocols that enable different applications or systems to communicate with each other.

In contrast to a user interface, which connects a computer to a human, an application programming interface connects computers or pieces of software to one other.

Organizations can share data with each other using APIs.

Restaurant and Bank Analogy



Restaurant and Bank Analogy



Application Programming Interface (API)

Private APIs vs Public APIs

Private APIs: A private API is only available to developers and users within a company. These APIs are typically used for internal communications between different components of a system.

Private APIs

When a program uses a database, it often uses a database driver or connector to interact with the database. The database driver provides a private API that allows the program to perform CRUD (Create, Read, Update, Delete) operations on the data stored in the database.

For example, in a web application, the backend code might use a database API to retrieve user information from the database, update a user's profile information, etc.

This database API is considered private because:

- It's not exposed to the public web
- It's only accessible within the program or application
- It's used for internal communication between the program and the database

Private APIs vs Public APIs

Public APIs: On the other hand, public APIs allow third-party developers to quickly access and merge data from one tool to another. A public API saves time for developers by allowing them to link their platform with already existing tools, minimizing the need to construct new services.

e.g. Google maps API, Third-party login, pay with PayPal/visa

Open API: Free to public

Use cases of API

1. Weather Data – Google fetches weather data from Weather Channel, Weather Underground, and AccuWeather.

Google “Ottawa Weather”

2. Login using third party authentication

3. Google Maps – Can you name some apps?

4. Address Fetching

API in action

<https://api.github.com/users/swapnilin>

The API **GET** request above retrieves information about the **GitHub** user swapnilin. This request uses the GitHub REST API to retrieve a JSON representation of the user's profile data, which includes information such as the user's name, location, email address (if publicly available), avatar URL, and a list of the user's public repositories.

This data can then be used by other applications or websites to display the user's profile information or to perform other operations related to the user's repositories or other GitHub data.

API Documentation

There are all kinds of companies out there that provide data through the use of their APIs. Data on sports, weather, finance, outer space, gifs, etc. Like a menu in a restaurant, companies provide documentation that explains **what data is available** and **how to ask** for and **get that data**.

<https://openweathermap.org/api>

<https://unsplash.com/developers>

<https://developers.gettyimages.com/docs/>

JSON

Our goal with making a get request is that we want to get some data back as a response. So, what data will we get? What is that data going to look like? What format will that data be in? There are really two main data formats you'll see: XML and JSON.

JSON stands for **JavaScript Object Notation**

JSON can get confusing as more and more key value pairs and layers of key value pairs are added.

JSON

```
{  
  "Title": "The Cuckoo's Calling"  
  "Author": "Robert Galbraith",  
  "Genre": "classic crime novel",  
  "Detail": {  
    "Publisher": "Little Brown"  
    "Publication_Year": 2013,  
    "ISBN-13": 9781408704004,  
    "Language": "English",  
    "Pages": 494  
  }  
  "Price": [  
    {  
      "type": "Hardcover",  
      "price": 16.65,  
    }  
    {  
      "type": "Kindle Edition",  
      "price": 7.03,  
    }  
  ]  
}
```

Diagram illustrating JSON syntax with annotations:

- Object Starts**: Indicated by an orange arrow pointing to the opening curly brace `{`.
- Object Starts**: Indicated by an orange arrow pointing to the opening curly brace `{` inside the `"Detail"` object.
- Value string**: Indicated by a blue arrow pointing to the string value `"Little Brown"`.
- Value number**: Indicated by a yellow arrow pointing to the numeric value `2013`.
- Object ends**: Indicated by an orange arrow pointing to the closing curly brace `}` of the `"Detail"` object.
- Array starts**: Indicated by a green arrow pointing to the opening square bracket `[` of the `"Price"` array.
- Object Starts**: Indicated by an orange arrow pointing to the opening curly brace `{` of the first object in the `"Price"` array.
- Object ends**: Indicated by an orange arrow pointing to the closing curly brace `}` of the first object in the `"Price"` array.
- Object Starts**: Indicated by an orange arrow pointing to the opening curly brace `{` of the second object in the `"Price"` array.
- Object ends**: Indicated by an orange arrow pointing to the closing curly brace `}` of the second object in the `"Price"` array.
- Array ends**: Indicated by a green arrow pointing to the closing square bracket `]` of the `"Price"` array.
- Object ends**: Indicated by an orange arrow pointing to the closing curly brace `}` of the main object.

<https://www.w3resource.com/JSON/introduction.php>

Parsing the data

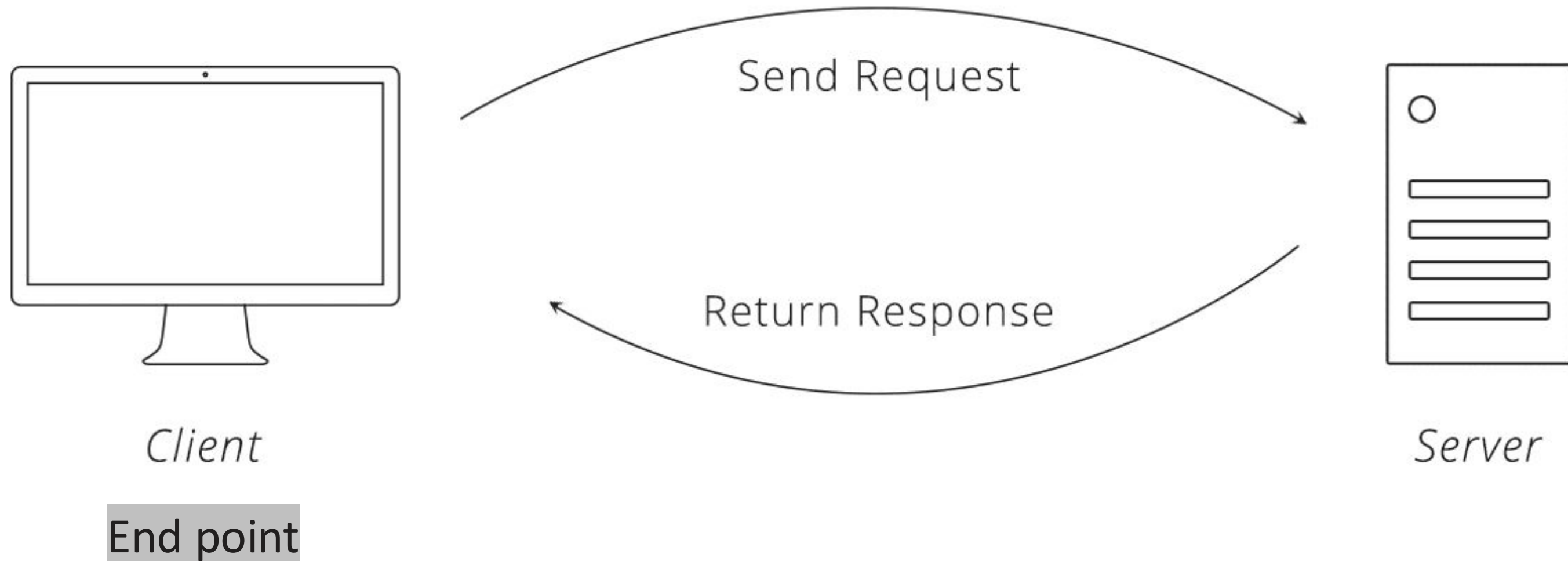
When data is sent in the response, it's typically sent as a string or readable stream depending on the method we use to send the request. We can think of the Data like a package being sent from the server to us and a string or readable stream is often the best, easiest way to send it. But in order to work with the data in JavaScript, we have to convert it to JSON.

This is kind of like opening the package, often referred to as **parsing**.

Types of API

1. REST API (Representational State Transfer) is a type of API architecture that uses HTTP requests to access and manipulate data. RESTful APIs typically use HTTP verbs (GET, POST, PUT, DELETE) to perform CRUD (Create, Read, Update, Delete) operations on resources, and responses are typically returned in a standardized format, such as JSON or XML.
2. GraphQL API: An example of a GraphQL API is the GitHub API, which enables developers to access information about repositories, pull requests, and other data on the GitHub platform.
3. WebSocket API - is a technology that enables persistent, bidirectional communication between a client (like a web browser) and a server. The choice of API depends on the specific needs of the application and the systems it needs to interact with.

Request and Response Cycle



Request options using HTTP protocol

HTTP Methods and Their Meaning

Method	Meaning
GET	Read data
POST	Insert data
PUT or PATCH	Update data, or insert if a new id
DELETE	Delete data

Web Scrapping

Web scraping can be a solution if a website doesn't provide an API or if the available API doesn't provide the required data or functionality.

Web Scrapping

Web scraping is the process of extracting data or information from websites or web pages automatically using a computer program or script.

Web scraping can be used for various purposes, such as gathering market intelligence, monitoring competitors' prices, collecting customer reviews, or any other use case where large amounts of data need to be collected from the web.

The term **scraping** is used for getting data off of a single page and **“crawling”** specifically refers to the act of moving from page to page.

Web Scrapping Python Libraries

1. BeautifulSoup: a popular library for parsing HTML and XML documents. It provides a simple interface for navigating and searching the document tree.
2. Scrapy: a framework for web scraping that provides powerful tools for crawling websites and extracting data. It supports parallel processing, asynchronous networking, and data storage.
3. Selenium: a web automation tool that can be used for scraping dynamic web pages and interacting with JavaScript-based content. It simulates user interaction with the web page and can be used to automate web browsers.

Let's do some demo